

AGEX Product Philosophy

1. 문서 개요

1.1 문서 목적

본 문서는 AGEX의 제품 철학과 설계 원칙을 정의한다.

Product Vision이 AGEX가 무엇을 지향하는지를 정의한다면, Product Philosophy는 AGEX를 어떤 기준으로 설계하고 개발하고 운영할 것인지를 규정한다.

본 문서는 다음 의사결정의 기준으로 사용한다.

- 제품 범위 결정
- 기능 우선순위 결정
- 시스템 구조 설계
- 기술 스택 선택
- Agent 실행 방식 정의
- Workflow 설계
- Knowledge 및 Memory 정책 수립
- Plugin 구조 설계
- Marketplace 운영 정책 수립
- 보안 및 권한 정책 정의
- 사용자 경험 설계
- 운영 정책 수립
- 확장 기능 검토
- 기술 부채 관리

1.2 적용 대상

본 문서는 AGEX를 설계, 개발, 검토, 운영 또는 확장하는 모든 역할에 적용한다.

- Product Owner
- Product Manager

- Chief Architect
- Enterprise Architect
- Software Architect
- AI Engineer
- Backend Engineer
- Frontend Engineer
- Platform Engineer
- DevOps Engineer
- Security Engineer
- Data Engineer
- QA Engineer
- Plugin Developer
- Application Builder
- Platform Operator

1.3 문서의 우선순위

AGEX의 기능이나 기술 설계가 본 문서의 철학과 충돌할 경우 다음 순서로 판단한다.

1. 제품 비전과 일치하는가.
2. 범용 플랫폼으로서 재사용 가능한가.
3. 특정 고객, 산업, 모델 또는 공급자에 종속되지 않는가.
4. Agent, Workflow, Knowledge, Memory, Plugin, Runtime의 일관성을 유지하는가.
5. 보안, 통제, 확장성 및 운영 가능성을 훼손하지 않는가.

위 기준을 충족하지 못하는 기능은 AGEX Core에 포함하지 않는다.

2. 제품 철학 선언

AGEX의 제품 철학은 다음 문장으로 정의한다.

AI는 단순한 응답 기능이 아니라, 목표를 이해하고 판단하고 협업하고 실행하는 운영 주체여야 한다.

AGEX는 AI를 부가 기능으로 취급하지 않는다.

AGEX에서 AI는 시스템의 일부 기능이 아니라 시스템을 구성하고 실행하는 핵심 주체이다.

AGEX는 다음 구조를 기본 전제로 한다.

- Agent는 실행 주체이다.
- Workflow는 실행 절차이다.
- Knowledge는 판단 근거이다.
- Memory는 지속성의 기반이다.
- Plugin은 외부 기능과의 연결 수단이다.
- Runtime은 실행을 통제하는 운영 환경이다.
- Marketplace는 확장 자산을 유통하는 생태계이다.
- Application Layer는 AGEX 기능을 조합하여 사용자에게 제공하는 표현 계층이다.

AGEX의 모든 제품 설계는 이 구조를 중심으로 수행한다.

3. 핵심 제품 철학

AGEX는 다음 핵심 철학을 따른다.

1. AI First
2. Agent Native
3. Workflow Native
4. Knowledge Native
5. Memory Native
6. Plugin Native
7. Runtime Centered
8. Model Agnostic

9. API First
10. Event Driven
11. Security by Design
12. Zero Trust
13. Cloud Native
14. Multi-Tenant by Default
15. Observable by Default
16. Extensible by Default
17. Automation with Control
18. Human Governed Autonomy
19. Everything is an Agent
20. Everything is Connectable
21. Everything is Automatable

각 철학은 단순한 표어가 아니라 제품과 기술 구조에 반영되어야 하는 설계 원칙이다.

4. AI First

4.1 정의

AI First는 기존 소프트웨어 구조에 AI 기능을 추가하는 방식이 아니라, 처음부터 AI가 판단과 실행에 참여하는 구조로 제품을 설계하는 원칙이다.

4.2 적용 기준

AGEX의 각 기능은 다음 질문을 기준으로 설계한다.

- AI가 이 기능의 실행에 어떤 역할을 하는가.
- AI가 입력을 이해하고 구조화할 수 있는가.
- AI가 실행 계획을 생성할 수 있는가.
- AI가 적절한 Agent, Workflow, Plugin 또는 Model을 선택할 수 있는가.

- AI가 실행 결과를 평가할 수 있는가.
- AI의 판단을 정책과 권한으로 통제할 수 있는가.

4.3 구현 원칙

AI First는 모든 기능을 LLM 호출로 구현한다는 의미가 아니다.

다음 영역은 결정적 로직으로 처리한다.

- 인증
- 권한 검증
- 정책 집행
- 자원 할당
- 상태 전이
- 트랜잭션 처리
- 비밀정보 관리
- 감사 기록
- 결제 및 과금
- 버전 관리
- 실행 제한
- Tenant 격리

AI는 판단과 생성에 활용하되, 시스템 통제는 명시적인 규칙과 정책으로 수행한다.

4.4 금지 원칙

다음 설계는 허용하지 않는다.

- AI의 판단 결과를 검증 없이 직접 실행
- 모델 응답을 시스템 상태로 즉시 반영
- 권한 확인 없이 Plugin 실행
- AI가 자체적으로 권한을 확대하는 구조
- 생성된 Workflow를 검증 없이 배포

- 모델 출력 형식에 시스템 안정성을 의존하는 구조
-

5. Agent Native

5.1 정의

Agent Native는 Agent를 부가 기능이 아니라 AGEX의 기본 실행 단위로 취급하는 원칙이다.

5.2 Agent의 역할

Agent는 다음 기능을 수행할 수 있다.

- 목표 해석
- 작업 분해
- 실행 계획 수립
- Knowledge 조회
- Memory 조회 및 기록
- Plugin 호출
- 다른 Agent와 협업
- Workflow 참여
- 실행 결과 평가
- 오류 원인 분석
- 재시도 경로 결정
- 결과 전달
- 상태 보고

5.3 Agent 구성 원칙

모든 Agent는 최소한 다음 정보를 가져야 한다.

- Agent ID
- Name

- Version
- Role
- Goal
- Instruction
- Capability
- Permission Scope
- Model Policy
- Tool Access
- Knowledge Access
- Memory Policy
- Execution Policy
- Resource Limit
- Timeout Policy
- Retry Policy
- Audit Policy
- Lifecycle State

5.4 Agent 독립성

Agent는 특정 Application Layer에 종속되어서는 안 된다.

Agent는 독립적으로 등록, 버전 관리, 배포, 호출 및 폐기할 수 있어야 한다.

동일한 Agent를 여러 Workflow와 Application Layer에서 재사용할 수 있어야 한다.

5.5 Agent 통제

Agent의 자율성은 무제한이 아니다.

Agent는 다음 범위 안에서만 행동할 수 있다.

- 할당된 Role
- 허용된 Capability

- 승인된 Plugin
 - 접근 가능한 Knowledge
 - 접근 가능한 Memory
 - Tenant 권한
 - 실행 예산
 - 시간 제한
 - 정책 규칙
 - 승인 상태
-

6. Workflow Native

6.1 정의

Workflow Native는 복합 작업을 Application 코드에 직접 구현하지 않고 독립적인 실행 정의로 관리하는 원칙이다.

6.2 Workflow 구성요소

Workflow는 다음 요소로 구성할 수 있다.

- Trigger
- Step
- Agent Task
- Plugin Task
- Function Task
- Condition
- Branch
- Loop
- Parallel Execution
- Join

- Approval
- Wait
- Event Subscription
- Timeout
- Retry
- Compensation
- Error Handler
- Output Mapping
- Completion Condition

6.3 Workflow 독립성

Workflow는 특정 화면, 특정 사용자 인터페이스 또는 특정 Application Layer에 종속되지 않아야 한다.

Workflow는 다음 방식으로 호출할 수 있어야 한다.

- API 호출
- Event 발생
- 사용자 요청
- Agent 호출
- Scheduler 실행
- 다른 Workflow 호출
- Plugin Callback

6.4 선언적 정의

Workflow는 가능한 한 선언적으로 정의한다.

Workflow 정의에는 최소한 다음 정보가 포함되어야 한다.

- Workflow ID
- Version

- Input Schema
- Output Schema
- Trigger
- Step Definitions
- Dependencies
- Permissions
- Error Policies
- Timeout Policies
- Retry Policies
- Resource Limits
- Audit Requirements

6.5 Workflow 실행 원칙

Workflow 실행은 재현 가능해야 한다.

동일한 입력과 동일한 버전의 구성요소가 사용된 경우 실행 경로를 추적할 수 있어야 한다.

모델 특성상 결과값이 달라질 수 있으나, 다음 정보는 반드시 기록되어야 한다.

- 사용된 Workflow 버전
- 사용된 Agent 버전
- 사용된 Plugin 버전
- 사용된 Model
- Prompt 및 정책 버전
- 실행 시점
- 입력 데이터
- 출력 데이터
- 상태 전이

- 오류 및 재시도 내역
-

7. Knowledge Native

7.1 정의

Knowledge Native는 지식 자산을 외부 참고 자료가 아니라 Agent 판단에 사용되는 공식 시스템 구성요소로 관리하는 원칙이다.

7.2 Knowledge의 범위

Knowledge는 다음 형태를 포함할 수 있다.

- 문서
- 구조화 데이터
- 정책
- 규칙
- 용어 정의
- 메타데이터
- 관계 정보
- API 기반 외부 데이터
- 계산된 정보
- 검증된 결과
- 버전이 관리되는 지식 패키지

7.3 Knowledge 관리 원칙

Knowledge는 다음 특성을 가져야 한다.

- 출처 식별
- 소유자 식별
- Tenant 식별
- 버전 관리

- 접근 권한
- 유효 기간
- 신뢰도
- 인덱싱 상태
- 갱신 시점
- 폐기 상태
- 검색 정책
- 인용 가능성

7.4 Knowledge와 원본 데이터

Knowledge는 원본 데이터와 구분하여 관리한다.

원본 데이터는 수집된 사실 또는 자료이며, Knowledge는 Agent가 사용할 수 있도록 정규화, 검증, 분류, 인덱싱 또는 관계화된 정보 자산이다.

7.5 Knowledge 사용 원칙

Agent가 Knowledge를 사용할 때는 다음 정보를 추적해야 한다.

- 조회한 Knowledge ID
- 사용한 버전
- 검색 조건
- 검색 결과
- 선택된 근거
- 결과에 미친 영향
- 출처
- 인용 여부

8. Memory Native

8.1 정의

Memory Native는 AI의 지속적인 실행과 맥락 유지를 위해 Memory를 시스템의 기본 기능으로 제공하는 원칙이다.

8.2 Memory 유형

AGEX는 Memory를 다음 유형으로 구분한다.

8.2.1 Working Memory

현재 실행 중인 Agent 또는 Workflow가 사용하는 임시 상태이다.

8.2.2 Session Memory

하나의 세션이나 연속된 상호작용 동안 유지되는 맥락이다.

8.2.3 Agent Memory

특정 Agent가 반복 실행 과정에서 사용하는 경험과 상태이다.

8.2.4 User Memory

사용자와의 상호작용을 지속하기 위해 저장되는 허용된 정보이다.

8.2.5 Tenant Memory

Tenant 내부에서 공유되는 설정, 실행 경험 또는 공통 맥락이다.

8.2.6 Episodic Memory

특정 사건이나 실행 단위에 대한 기록이다.

8.2.7 Semantic Memory

사실, 개념, 관계 및 요약된 정보이다.

8.2.8 Procedural Memory

특정 작업을 수행하는 방법, 실행 패턴 및 절차 정보이다.

8.3 Memory 저장 원칙

Memory는 모든 정보를 무조건 저장하지 않는다.

저장 여부는 다음 기준에 따라 결정한다.

- 명시적 저장 정책
- 사용자 동의

- Tenant 정책
- 데이터 분류
- 보안 등급
- 보존 기간
- 재사용 가치
- 정확성
- 중복 여부
- 삭제 요청
- 법적 또는 계약상 제한

8.4 Memory 안전성

Agent는 Memory를 임의로 영구 저장할 수 없다.

영구 Memory 저장은 정책 엔진의 검증을 받아야 한다.

민감한 정보는 저장 제한, 암호화, 마스킹 또는 비저장 정책을 적용할 수 있어야 한다.

8.5 Memory 수정과 삭제

Memory는 다음 작업을 지원해야 한다.

- 조회
- 추가
- 수정
- 병합
- 요약
- 만료
- 폐기
- 삭제
- 복구
- 출처 추적

- 접근 이력 확인
-

9. Plugin Native

9.1 정의

Plugin Native는 외부 기능, 서비스, 데이터 또는 실행 도구와의 연결을 표준 Plugin 구조로 관리하는 원칙이다.

9.2 Plugin의 역할

Plugin은 다음 기능을 수행할 수 있다.

- 외부 API 호출
- 외부 데이터 조회
- 데이터 생성 및 수정
- 명령 실행
- 파일 처리
- Event 수신
- Event 발행
- 인증 처리
- 외부 시스템 상태 확인
- 비동기 작업 요청
- Callback 수신

9.3 Plugin 명세

모든 Plugin은 최소한 다음 정보를 제공해야 한다.

- Plugin ID
- Name
- Version
- Provider

- Description
- Capability
- Input Schema
- Output Schema
- Authentication Type
- Required Permissions
- Network Policy
- Timeout
- Rate Limit
- Error Codes
- Retry Policy
- Data Handling Policy
- Audit Policy
- Compatibility
- Lifecycle State

9.4 Plugin 격리

Plugin 실행은 AGEX Core와 분리된 실행 경계를 가져야 한다.

Plugin의 오류가 Runtime 전체 장애로 전파되지 않아야 한다.

Plugin은 할당된 권한, 네트워크, 자원 및 데이터 범위 안에서만 실행되어야 한다.

9.5 Plugin 신뢰 등급

Plugin은 검증 상태에 따라 등급을 구분할 수 있다.

- Internal
- Verified
- Certified
- Community

- Restricted
- Deprecated
- Blocked

신뢰 등급에 따라 설치, 실행, 권한 승인 및 Marketplace 노출 정책을 다르게 적용한다.

10. Runtime Centered

10.1 정의

Runtime Centered는 Agent와 Workflow의 실제 실행을 개별 Application Layer가 아니라 공통 Runtime에서 관리하는 원칙이다.

10.2 Runtime 책임

Runtime은 다음을 담당한다.

- 실행 요청 수신
- 입력 검증
- 권한 확인
- 정책 평가
- 실행 계획 생성
- Agent 할당
- Workflow 상태 관리
- Model 호출
- Plugin 실행
- Memory 연결
- Knowledge 연결
- 자원 할당
- Timeout 관리
- Retry 관리

- 오류 처리
- 상태 저장
- 로그 기록
- 비용 측정
- 결과 반환
- 실행 종료
- 복구 처리

10.3 실행 일관성

모든 Agent와 Workflow는 동일한 Runtime 규칙을 따라야 한다.

Application Layer가 Runtime을 우회하여 Agent, Model 또는 Plugin을 직접 실행하는 구조는 허용하지 않는다.

10.4 Runtime 정책 우선

Runtime은 Agent의 지시보다 상위 권한을 가진다.

Agent의 요청이 정책, 권한, 자원 제한 또는 보안 규칙을 위반할 경우 Runtime은 실행을 거부해야 한다.

11. Model Agnostic

11.1 정의

Model Agnostic은 특정 모델 또는 모델 공급자를 제품의 고정 기반으로 전제하지 않는 원칙이다.

11.2 모델 추상화

AGEX는 모델을 공통 Model Provider Interface를 통해 호출한다.

모델별 차이는 Adapter 계층에서 처리한다.

11.3 지원 대상

AGEX는 다음 유형의 모델을 수용할 수 있어야 한다.

- 외부 API 기반 모델

- 자체 호스팅 모델
- 상용 모델
- 오픈 모델
- 텍스트 모델
- 이미지 모델
- 음성 모델
- 임베딩 모델
- 재순위 모델
- 분류 모델
- 멀티모달 모델

11.4 모델 선택 정책

모델은 다음 조건에 따라 선택할 수 있다.

- Agent 정책
- Task 유형
- 입력 형식
- 출력 품질 요구
- 지연시간
- 비용 제한
- 보안 등급
- 데이터 위치
- Context 길이
- 모델 가용성
- Tenant 설정
- 규제 조건

11.5 모델 교체 가능성

모델 변경이 Agent, Workflow 또는 Application Layer의 전체 재개발로 이어져서는 안 된다.

모델별 Prompt 차이와 출력 차이는 Template, Adapter 및 Validation 계층에서 흡수한다.

12. API First

12.1 정의

API First는 모든 핵심 기능을 사용자 인터페이스보다 먼저 API 계약으로 정의하는 원칙이다.

12.2 API 대상

다음 기능은 API를 통해 접근할 수 있어야 한다.

- Agent 관리
- Agent 실행
- Workflow 관리
- Workflow 실행
- Knowledge 관리
- Knowledge 조회
- Memory 관리
- Plugin 관리
- Plugin 실행
- Model 관리
- Runtime 상태 조회
- Event 발행 및 구독
- Tenant 관리
- 사용자 및 권한 관리
- Marketplace 관리
- 로그 및 감사 조회

- 비용 및 사용량 조회

12.3 API 설계 원칙

API는 다음 특성을 가져야 한다.

- 명확한 Resource 모델
- 일관된 URI 또는 Method 구조
- 버전 관리
- 입력 및 출력 Schema
- 오류 코드 체계
- 인증과 권한
- Idempotency
- Pagination
- Filtering
- Sorting
- Rate Limiting
- Trace ID
- 감사 기록
- 하위 호환성 정책

12.4 UI 종속성 금지

사용자 인터페이스 전용 내부 기능을 만들지 않는다.

UI는 공식 API를 사용하는 Client 중 하나로 취급한다.

13. Event Driven

13.1 정의

Event Driven은 시스템 상태 변화와 실행 요청을 Event를 통해 전달하고 처리할 수 있도록 설계하는 원칙이다.

13.2 Event의 역할

Event는 다음 상황에 사용한다.

- Agent 실행 요청
- Workflow 시작
- Workflow 상태 변경
- Plugin 결과 수신
- Knowledge 갱신
- Memory 갱신
- 정책 변경
- 사용자 동작
- Tenant 상태 변경
- Marketplace 배포
- 오류 발생
- 자원 임계치 도달
- 승인 완료
- 외부 Callback 수신

13.3 Event 명세

모든 Event는 최소한 다음 필드를 가져야 한다.

- Event ID
- Event Type
- Event Version
- Source
- Tenant ID
- Subject
- Timestamp

- Correlation ID
- Causation ID
- Payload
- Security Context
- Trace Context

13.4 Event 처리 원칙

Event 처리는 다음을 보장해야 한다.

- 중복 처리 방지
- 순서 보장 범위 정의
- 재처리 가능성
- 실패 Event 격리
- Dead Letter 처리
- 소비자 추적
- Schema 버전 관리
- 보존 정책
- 감사 가능성

14. Security by Design

14.1 정의

Security by Design은 보안을 사후 기능이 아니라 제품 설계 단계부터 모든 구성요소에 포함하는 원칙이다.

14.2 기본 적용 영역

보안은 다음 영역에 적용한다.

- Identity
- Authentication

- Authorization
- Tenant Isolation
- Agent Permission
- Plugin Permission
- Knowledge Access
- Memory Access
- Model Access
- Secret Management
- Network Control
- Data Encryption
- Audit Logging
- Policy Enforcement
- Runtime Isolation
- Supply Chain Security
- Marketplace Verification

14.3 최소 권한

모든 사용자, Agent, Plugin, Workflow 및 Service는 필요한 최소 권한만 가져야 한다.

기본값은 허용이 아니라 거부이다.

14.4 실행 전 검증

모든 실행은 다음 검증을 통과해야 한다.

- 호출 주체 확인
- Tenant 확인
- Resource 권한 확인
- Agent 권한 확인
- Plugin 권한 확인

- 데이터 접근 권한 확인
 - 정책 위반 여부 확인
 - 비용 한도 확인
 - 실행 환경 확인
-

15. Zero Trust

15.1 정의

Zero Trust는 내부와 외부로 구분하여 자동으로 신뢰하지 않고, 모든 요청과 실행을 매번 검증하는 원칙이다.

15.2 적용 대상

다음 구성요소는 기본적으로 신뢰하지 않는다.

- 사용자
- Agent
- Plugin
- Model
- 외부 API
- Marketplace Package
- Tenant 간 요청
- 내부 Service 간 요청
- Event Producer
- Event Consumer
- 저장된 데이터
- 생성된 Workflow
- Agent가 생성한 실행 계획

15.3 지속 검증

권한은 로그인 시 한 번만 확인하지 않는다.

중요한 자원 접근과 실행 시점마다 정책을 다시 평가한다.

16. Cloud Native

16.1 정의

Cloud Native는 AGEX를 분산 환경, 탄력적 확장, 자동 복구 및 선언적 배포를 전제로 설계하는 원칙이다.

16.2 적용 기준

AGEX는 다음을 지원해야 한다.

- Container 기반 배포
- 독립 Service 배포
- 수평 확장
- Stateless Service
- 외부 상태 저장소
- 선언적 인프라
- 자동 복구
- Rolling Update
- Canary Deployment
- Health Check
- Service Discovery
- Configuration 분리
- Secret 분리
- 분산 추적
- 중앙 로그

16.3 특정 Cloud 종속 금지

AGEX Core는 특정 Cloud Provider의 전용 기능에 직접 종속되지 않는다.

필요한 경우 Provider Adapter를 통해 연동한다.

17. Multi-Tenant by Default

17.1 정의

Multi-Tenant by Default는 AGEX의 모든 Resource와 실행을 Tenant 경계 안에서 관리하는 원칙이다.

17.2 Tenant 적용 대상

다음 Resource는 Tenant를 명시적으로 식별해야 한다.

- User
- Agent
- Workflow
- Knowledge
- Memory
- Plugin Installation
- Model Configuration
- Runtime Execution
- Event
- Policy
- Secret
- Usage
- Billing
- Audit Log
- Marketplace Subscription

17.3 Tenant 격리

Tenant 간에는 다음이 분리되어야 한다.

- 데이터
- Memory
- Knowledge
- Secret
- Agent
- Workflow
- Plugin 설정
- 실행 기록
- 비용
- 권한
- 정책
- 로그

17.4 공유 자산

Marketplace 자산처럼 공유 가능한 Resource도 Tenant 설치 또는 사용 시 별도의 권한과 설정을 가져야 한다.

18. Observable by Default

18.1 정의

Observable by Default는 모든 주요 실행을 상태, 로그, 지표 및 Trace를 통해 확인할 수 있도록 하는 원칙이다.

18.2 관측 대상

다음 항목을 관측할 수 있어야 한다.

- Agent 실행 상태
- Workflow 진행 상태
- Step별 처리 시간

- Model 호출 횟수
- Token 사용량
- 비용
- Plugin 호출 상태
- 오류
- 재시도
- Timeout
- Queue 지연
- Resource 사용량
- Knowledge 조회 결과
- Memory 접근
- 정책 거부
- 권한 오류
- Tenant별 사용량

18.3 추적성

하나의 요청은 전체 실행 과정에서 동일한 Correlation ID와 Trace Context로 추적할 수 있어야 한다.

18.4 설명 가능성

AI 판단의 내부 사고 과정을 그대로 저장하는 것을 전제로 하지 않는다.

대신 다음 정보를 기록한다.

- 사용한 입력
- 선택된 정책
- 사용된 Knowledge
- 호출된 Plugin
- 선택된 Model

- 실행 결과
 - 검증 결과
 - 실패 원인
 - 승인 내역
-

19. Extensible by Default

19.1 정의

Extensible by Default는 새로운 기능을 Core 수정 없이 확장할 수 있도록 설계하는 원칙이다.

19.2 확장 단위

AGEX의 주요 확장 단위는 다음과 같다.

- Agent
- Workflow
- Plugin
- Model Adapter
- Knowledge Connector
- Memory Provider
- Event Handler
- Policy Module
- Marketplace Package
- SDK Extension

19.3 확장 방식

확장은 다음 표준을 따라야 한다.

- 명시적 Interface
- Versioned Contract

- Schema Validation
- Compatibility Rule
- Permission Declaration
- Lifecycle Management
- Error Contract
- Audit Support
- Observability Support
- Resource Limit

19.4 Core 보호

확장 기능이 Core 데이터베이스에 직접 접근하거나 내부 Service를 우회하는 방식은 허용하지 않는다.

모든 확장은 공식 API, Event 또는 Extension Interface를 사용해야 한다.

20. Automation with Control

20.1 정의

Automation with Control은 가능한 작업을 자동 실행하되, 자동화 범위를 정책, 권한, 승인 및 감사 체계로 통제하는 원칙이다.

20.2 자동화 대상

자동화는 다음 방식으로 수행할 수 있다.

- Event Trigger
- Schedule Trigger
- API Trigger
- Agent Decision
- Workflow Condition
- External Callback
- State Change

20.3 통제 수단

자동화는 다음 수단으로 통제한다.

- Permission
- Policy
- Approval
- Cost Limit
- Rate Limit
- Time Limit
- Scope Limit
- Data Classification
- Environment Restriction
- Kill Switch
- Audit Log

20.4 중단 가능성

모든 장기 실행 Workflow와 Agent는 중지, 일시정지, 재개 및 강제 종료가 가능해야 한다.

21. Human Governed Autonomy

21.1 정의

Human Governed Autonomy는 AI가 자율적으로 판단하고 실행할 수 있지만, 최종 통제권은 인간과 명시적 정책에 있어야 한다는 원칙이다.

21.2 자율성 수준

AGEX는 Agent의 자율성 수준을 구분하여 관리해야 한다.

- Level 0: 응답만 생성
- Level 1: 실행 계획 제안
- Level 2: 승인 후 실행

- Level 3: 제한된 범위 내 자동 실행
- Level 4: 정책 기반 지속 실행
- Level 5: 다중 Agent 자율 협업

각 수준은 권한, 위험도, 비용 및 감사 요구사항에 따라 설정한다.

21.3 인간 개입 지점

다음 상황에서는 인간 승인 또는 검토를 요구할 수 있다.

- 높은 위험의 실행
- 외부 데이터 수정
- 비용 임계치 초과
- 민감정보 접근
- 권한 변경
- 새로운 Plugin 설치
- Marketplace 배포
- 중요한 Workflow 변경
- 정책 예외 요청
- 복구 불가능한 작업

21.4 긴급 통제

관리자는 다음 기능을 가져야 한다.

- Agent 즉시 중지
- Workflow 즉시 중지
- Plugin 차단
- Model 차단
- Tenant 실행 정지
- Credential 폐기
- Event 소비 중단

- 실행 Queue 비우기
 - 정책 긴급 변경
-

22. Everything is an Agent

22.1 정의

Everything is an Agent는 판단, 목표, 권한 및 실행 능력을 가진 구성요소를 공통 Agent 모델로 표현할 수 있다는 철학이다.

22.2 적용 범위

다음 특성을 가진 구성요소는 Agent로 설계할 수 있다.

- 목표를 가진다.
- 입력을 해석한다.
- 상태를 유지한다.
- 선택을 수행한다.
- 도구를 사용한다.
- 결과를 평가한다.
- 다른 Agent와 협업한다.

22.3 제한

모든 Service, API 또는 단순 함수까지 Agent로 구현한다는 의미는 아니다.

결정적이고 단순한 처리에는 일반 Service 또는 Function을 사용한다.

Agent는 판단과 적응이 필요한 영역에만 적용한다.

23. Everything is Connectable

23.1 정의

Everything is Connectable은 AGEX의 주요 구성요소가 표준 Interface를 통해 상호 연결 될 수 있어야 한다는 철학이다.

23.2 연결 대상

다음 구성요소는 연결 가능해야 한다.

- Agent와 Agent
- Agent와 Workflow
- Agent와 Knowledge
- Agent와 Memory
- Agent와 Plugin
- Agent와 Model
- Workflow와 Workflow
- Workflow와 Event
- Plugin과 외부 시스템
- Application Layer와 API
- Marketplace와 Runtime

23.3 연결 표준

연결은 다음 방식을 사용한다.

- API
- Event
- Message
- Plugin Contract
- Workflow Invocation
- Function Call
- SDK
- Callback

직접적인 내부 의존성보다 계약 기반 연결을 우선한다.

24. Everything is Automatable

24.1 정의

Everything is Automatable은 반복 가능하고 규칙 또는 조건으로 표현할 수 있는 실행은 자동화할 수 있어야 한다는 철학이다.

24.2 자동화 조건

다음 요소가 정의될 수 있으면 자동화 대상으로 본다.

- Trigger
- Input
- Permission
- Execution Step
- Decision Rule
- Output
- Failure Policy
- Completion Condition

24.3 자동화 금지 영역

다음 작업은 명시적 정책 없이 자동화하지 않는다.

- 권한 확대
- 보안 정책 해제
- 영구 삭제
- Credential 생성 또는 노출
- Tenant 간 데이터 이동
- 비가역적 외부 작업
- 비용 제한 해제
- 감사 기록 삭제
- Plugin 신뢰 등급 상향
- Marketplace 검증 우회

25. 단순성 원칙

25.1 복잡성의 통제

AGEX는 복잡한 기능을 제공하지만, 불필요한 구조적 복잡성을 허용하지 않는다.

모든 구성요소는 명확한 책임을 가져야 한다.

25.2 단일 책임

각 Service, Module, Agent, Plugin 및 Workflow Step은 하나의 명확한 책임을 가져야 한다.

25.3 명시성

암묵적 동작보다 명시적 설정을 우선한다.

다음 사항은 명시적으로 정의되어야 한다.

- 권한
- 입력
- 출력
- 실패 처리
- 재시도
- Timeout
- 비용 제한
- 데이터 접근
- Memory 저장
- Knowledge 사용
- Plugin 사용

25.4 기본값

기본값은 안전하고 예측 가능해야 한다.

- 기본 권한은 최소 권한

- 기본 네트워크는 차단
 - 기본 데이터 공유는 비활성화
 - 기본 장기 Memory 저장은 비활성화
 - 기본 외부 실행은 승인 필요
 - 기본 로그는 활성화
 - 기본 감사는 활성화
-

26. 예측 가능성과 재현성

26.1 예측 가능성

Agent와 Workflow는 실행 전 예상되는 권한, 자원, 비용, 시간 및 외부 영향 범위를 확인할 수 있어야 한다.

26.2 재현성

모델의 확률적 특성에도 불구하고 실행 구성은 재현 가능해야 한다.

다음 요소를 기록한다.

- Agent 버전
- Workflow 버전
- Plugin 버전
- Model 및 설정
- Prompt Template 버전
- Knowledge 버전
- 정책 버전
- 입력
- 실행 환경
- Seed 지원 여부
- Timestamp

26.3 변경 통제

운영 중인 구성요소는 직접 수정하지 않는다.

변경 시 새로운 버전을 생성하고 배포 절차를 거친다.

27. 실패를 전제로 한 설계

27.1 실패 가능성

AGEX는 다음 실패가 발생할 수 있음을 전제로 한다.

- Model 응답 오류
- Model 응답 지연
- Plugin 장애
- 외부 API 장애
- Network 장애
- 데이터 오류
- 정책 거부
- 권한 변경
- Event 중복
- Workflow Step 실패
- Agent 판단 오류
- 자원 부족
- Timeout
- 비용 제한 초과

27.2 실패 처리 원칙

모든 주요 실행에는 다음을 정의한다.

- 실패 조건
- 재시도 조건

- 최대 재시도 횟수
- 재시도 간격
- 대체 실행 경로
- 보상 작업
- 사용자 통지
- 운영자 통지
- Dead Letter 처리
- 수동 복구 방법

27.3 부분 실패

분산 실행에서는 전체 성공 또는 전체 실패만을 전제로 하지 않는다.

Step별 상태와 부분 결과를 저장하고 복구할 수 있어야 한다.

28. 비용 인식 설계

28.1 비용의 범위

AGEX는 다음 비용을 측정할 수 있어야 한다.

- Model 호출 비용
- Token 사용량
- Plugin 사용 비용
- Compute 사용량
- Storage 사용량
- Network 사용량
- Workflow 실행 비용
- Tenant별 비용
- Agent별 비용
- Application Layer별 비용

28.2 비용 통제

다음 제한을 설정할 수 있어야 한다.

- 요청별 한도
- Agent별 한도
- Workflow별 한도
- Tenant별 한도
- 기간별 한도
- Model별 한도
- Plugin별 한도

28.3 비용 기반 실행 정책

동일한 작업을 수행할 수 있는 여러 Model이나 Plugin이 있을 경우 비용, 품질 및 지연 시간을 기준으로 선택할 수 있어야 한다.

29. 데이터 소유권과 통제

29.1 데이터 소유권

Tenant가 생성하거나 제공한 데이터의 소유권은 해당 Tenant에 있다.

29.2 플랫폼 사용 범위

AGEX는 계약과 정책에 명시된 범위 안에서만 데이터를 처리한다.

29.3 학습 사용 제한

Tenant 데이터는 명시적 허용 없이 모델 학습, 공용 Knowledge 또는 다른 Tenant의 Memory에 사용하지 않는다.

29.4 삭제 가능성

Tenant는 정책과 보존 의무 범위 안에서 다음 데이터의 삭제를 요청할 수 있어야 한다.

- Knowledge
- Memory
- 실행 기록

- 업로드 데이터
 - 생성 결과
 - Plugin 데이터
 - 사용자 정보
-

30. 개발 철학

30.1 계약 우선 개발

구현 전 다음 계약을 먼저 정의한다.

- Domain Model
- API Contract
- Event Contract
- Schema
- Permission
- Error Code
- State Transition
- Version Policy

30.2 테스트 가능성

모든 핵심 구성요소는 자동 테스트가 가능해야 한다.

- Unit Test
- Contract Test
- Integration Test
- Security Test
- Load Test
- Failure Test
- Recovery Test

- Tenant Isolation Test
- Agent Evaluation
- Workflow Replay Test

30.3 점진적 확장

초기부터 모든 기능을 완성하려 하지 않는다.

Core Contract를 먼저 안정화하고, 구성요소를 독립적으로 확장한다.

30.4 기술 부채 통제

임시 구현은 반드시 다음 정보를 기록한다.

- 임시 적용 이유
- 영향 범위
- 제거 조건
- 대체 설계
- 해소 목표 버전

31. 제품 철학 검증 질문

새로운 기능 또는 구조를 AGEX에 포함하기 전 다음 질문을 검토한다.

1. 이 기능은 AGEX Core에 필요한가.
2. 특정 Application Layer에만 필요한 기능은 아닌가.
3. 특정 고객이나 산업에 종속되는가.
4. Agent 중심 구조와 일치하는가.
5. Workflow로 조합 가능한가.
6. Knowledge와 Memory의 책임이 구분되는가.
7. Plugin을 통해 확장 가능한가.
8. Runtime에서 통제 가능한가.
9. Model을 교체할 수 있는가.

10. API와 Event로 접근 가능한가.
11. Tenant가 명확히 구분되는가.
12. 권한과 정책이 정의되어 있는가.
13. 실행 상태를 관측할 수 있는가.
14. 실패 시 복구 가능한가.
15. 버전 관리가 가능한가.
16. 비용을 측정하고 제한할 수 있는가.
17. 데이터 소유권을 침해하지 않는가.
18. 인간의 통제권을 유지하는가.
19. Core 수정 없이 확장 가능한가.
20. Product Vision과 일치하는가.

하나 이상의 핵심 질문에 부합하지 않는 경우 설계를 수정하거나 AGEX Core에서 제외한다.

32. 제품 철학의 최종 정의

AGEX는 AI를 기능으로 추가하는 제품이 아니라 AI를 실행 주체로 수용하는 운영체제이다.

AGEX는 Agent가 목표를 수행하고, Workflow가 실행을 조정하며, Knowledge가 판단 근거를 제공하고, Memory가 맥락과 경험을 유지하며, Plugin이 외부 세계와 연결하고, Runtime이 모든 실행을 통제하는 구조를 따른다.

AGEX의 자율성은 통제되지 않은 자유가 아니다.

모든 자율 실행은 권한, 정책, 비용, 보안, 감사 및 인간의 통제 범위 안에서 이루어져야 한다.

AGEX의 확장성은 무제한적인 내부 수정이 아니다.

모든 확장은 명시적 계약, 표준 Interface, 버전 관리 및 격리된 실행 구조를 통해 이루어져야 한다.

AGEX의 핵심 철학은 다음과 같이 최종 요약한다.

- AI는 시스템의 중심 실행 주체이다.
- Agent는 기본 실행 단위이다.
- Workflow는 실행을 구조화한다.
- Knowledge는 판단 근거를 제공한다.
- Memory는 지속성을 제공한다.
- Plugin은 외부 기능을 연결한다.
- Runtime은 모든 실행을 통제한다.
- Model은 교체 가능한 자원이다.
- API와 Event는 연결의 기본 방식이다.
- 보안과 Tenant 격리는 기본 조건이다.
- 자동화는 통제 가능한 범위에서 수행한다.
- 인간은 최종 통제권을 유지한다.
- 모든 핵심 자산은 재사용 가능해야 한다.
- 모든 실행은 관측하고 감사할 수 있어야 한다.
- 모든 확장은 Core를 보호하면서 이루어져야 한다.

본 문서는 AGEX의 모든 제품 설계, 기술 설계, 개발, 배포, 운영 및 확장 의사결정에 적용하는 공식 Product Philosophy로 사용한다.